

MEDHAVA

Working with Claude, Codex and Grok

Read this before your next session rather than when a limit runs out.

The one thing that makes switching possible

The project lives in the repository, not in the conversation.

Everything — the code, the registers of what exists, the tests, the gates that refuse a false claim, the record of every verified run — is in files. Nothing important lives only in a chat window. That is why closing this session costs nothing, and it is worth protecting: the moment something important exists only in a conversation, switching tools becomes expensive.

So: **commit often**, and never let a decision live only in a chat.

Starting a new session, with any tool

Two things, in this order:

1. Give it the project — the repository, or the archive.
2. Paste `MEDHAVA_BOS_PROMPT.md` as the first message.

That file already exists, is already part of the build, and already states what the project is, what exists, what does not, and the rules the work must follow. It is the answer to "how do I start a new window". For customer-specific work, paste the prompt file that ships inside the tenant archive instead — that archive's own `START_HERE` names it. Keeping the customer's name out of the product's own handover is the same rule that lets this system carry a second business later without rewriting.

Then say what you want done. You do not need to re-explain the project.

When one runs out mid-project

Nothing is lost, and there is nothing to migrate.

Step	What you do
1	Make sure the last work is committed
2	Open the same project folder in the other tool
3	Paste <code>MEDHAVA_BOS_PROMPT.md</code>
4	Ask it to run <code>npm test</code> before changing anything

Step 4 is the important one. It tells you whether the project is in a good state before the new tool touches it — and it tells the new tool the same thing. If the suite fails, that is where the next session starts, whatever anybody intended to work on.

Which tool for which work

Only two of these rows are mine to claim; the third is your own measurement and is recorded as yours.

Work	Tool	Why
Editing across a large codebase	Codex, Cursor	They work inside the whole project, which is what most building is
Reasoning, design, documents, tracing a subtle defect	Claude	Longer chains of reasoning about consequences
Excel and reporting	Grok	Your measurement — you found it faster and more accurate than the alternatives for this work

Use whichever suits the task. The rule below is what keeps that safe.

The rule that keeps every tool honest

Whatever made the change, `npm test` must pass, and the claim must be recorded.

The gates do not know which tool edited a file and do not care. That is the entire point of them. Today 89 of 302 rules are proven by a test that runs, and a capability cannot be marked as working unless the command proving it is on record in `docs/verification/EVIDENCE.md` — which is then re-run to check it still holds.

So the question to ask any tool, always:

Which command did you run, and what did it print?

A summary is not an answer. Output is. A tool that cannot make the suite pass has not finished, whatever it says in the chat — and you do not need to read the code to hold it to that.

Checking this project with another tool

You said you would cross-check with Codex and Cursor. Good — here is how, in a way that does not depend on trusting anybody’s summary. Open the project in the other tool and ask it to run these, then compare what it reports against what you were told:

Command	What it answers
<code>npm test</code>	does everything still pass
<code>npm run test:product</code>	does the product work with no customer installed
<code>node brand/site/checkregistry.js --summary</code>	what actually runs, and what only exists on paper
<code>node brand/site/checkaudit.js --summary</code>	the score and the maturity level, recomputed
<code>node tools/evidence.js --check</code>	re-runs every recorded claim and reports any that no longer hold

The last one is the one to use if you ever suspect you are being told something flattering. It does not read anybody’s summary — it runs the commands again.

Every technical word above, in plain language

3 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

tenant

One business using the platform. Its people, its data and its settings are its own.

Us building mein ek office. Aapka office, aapka saamaan, aapka taala.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

Medhava · One business operating system · Any industry